

AE646 — Final Report (Stage 3)

Operator Learning for Parametric Darcy Flow:
Fourier Neural Operator vs. MLP Baseline with Physical Interpretation

Team PINNacles
Aryamann Srivastava
Department of Aerospace Engineering, IIT Kanpur

November 2026

Abstract

We compare Fourier Neural Operator (FNO) and a fully-connected MLP baseline as surrogate solvers for the 2D parametric Darcy flow equation $-\nabla \cdot (\kappa \nabla u) = f$, trained on 900 samples from the PDEBench benchmark at 64×64 resolution. FNO (4.7 M parameters) achieves a mean relative L_2 error of 0.052 on 200 held-out test samples, 36% lower than the MLP (0.082, 42 M parameters) at $9 \times$ lower parameter count. A scaled FNO (78.8 M parameters) further reduces error to 0.046. Both models infer in < 1 ms per sample, orders of magnitude faster than the FDM reference (1030 ms). FNO’s spectral weights enable zero-shot super-resolution: the same trained model evaluated at 128×128 attains mean error 0.059 without retraining. Component profiling reveals that spectral operations (FFT, complex multiply, IFFT) account for 68.4% of FNO’s compute time, with complex multiplication alone at 41%.

Contents

1	Introduction	3
2	Background	3
2.1	Darcy Flow and the Solution Operator	3
2.2	Fourier Neural Operator	3
3	Dataset and Preprocessing	4
3.1	PDEBench	4
3.2	Preprocessing Pipeline	4
3.3	Why Downsample?	4
3.4	Exploratory Data Analysis	4
4	Model Architectures	5
4.1	MLP Baseline	5
4.2	FNO Architecture	5
4.3	Key Architectural Differences	5
5	Training	5
6	Results	6
6.1	Test-Set Performance (64×64)	6
6.2	Analysis	7
6.3	MLP Baseline Ablation	7

7	Zero-Shot Super-Resolution	8
8	Inference Speed and Component Profiling	9
8.1	End-to-End Inference Speed	9
8.2	FNO Layer-wise Profiling	9
9	Physical Interpretation	10
9.1	Solution Structure and Model Behaviour	10
9.2	High-Error Cases	11
9.3	Zero-Shot Generalisation	11
10	Conclusion	11

1 Introduction

Scientific computing routinely requires solving parameterised PDEs over large parameter ensembles — for uncertainty quantification, optimal control, or inverse problems. Classical numerical solvers (FDM, FEM) are accurate but expensive: each new parameter configuration demands a full solve. Neural operator approaches [1, 3] learn a *solution operator* $\mathcal{G}^\dagger : \kappa \mapsto u$ directly from data, enabling amortised query cost after a one-time training phase.

The 2D Darcy flow equation is a canonical test bed:

$$-\nabla \cdot (\kappa(x, y) \nabla u(x, y)) = f(x, y), \quad (x, y) \in [0, 1]^2, \quad (1)$$

with $u = 0$ on $\partial\Omega$ and constant forcing $f = 1$. The permeability $\kappa \in \{0.1, 1.0\}$ is piecewise-constant on a 64×64 grid, varying between samples. The solution u is smooth given smooth κ , but exhibits sharp gradients near κ interfaces.

Contributions.

1. Full training, evaluation, and comparison of FNO and MLP on PDEBench Darcy flow.
2. Zero-shot super-resolution evaluation at $2\times$ training resolution.
3. Layer-wise computational profiling with correct GPU synchronisation.
4. Physical interpretation of error patterns, quantified against real κ -heterogeneity measures rather than asserted from visual inspection.
5. A real ablation over MLP baseline depth and optimiser choice, justifying the reported baseline configuration with measured numbers.

2 Background

2.1 Darcy Flow and the Solution Operator

Equation (1) describes pressure-driven flow through a porous medium. The solution operator $\mathcal{G}^\dagger$ maps the permeability field $\kappa \in L^\infty(\Omega)$ to the pressure $u \in H_0^1(\Omega)$. This operator is nonlinear (even though (1) is linear in u) because κ enters multiplicatively. For piecewise-constant κ , the solution is smooth away from κ -interfaces and exhibits $O(1)$ gradients at interfaces.

2.2 Fourier Neural Operator

Li et al. [1] proposed to discretise the Fourier integral operator:

$$(\mathcal{K}(a; \phi)v)(x) = \int_{\Omega} \kappa_{\phi}(x, y) v(y) dy \quad (2)$$

by truncating its Fourier expansion to the lowest $k_{\max}$ modes. The resulting *SpectralConv2d* layer computes:

$$\hat{v}_{j,l} = \text{FFT}(v)_{j,l}, \quad j \leq k_{\max}, l \leq k_{\max} \quad (3)$$

$$\hat{w}_{j,l} = R_{j,l} \hat{v}_{j,l}, \quad R \in \mathbb{C}^{C_{\text{out}} \times C_{\text{in}} \times k_{\max} \times k_{\max}} \quad (4)$$

$$w = \text{IFFT}(\hat{w}), \quad (5)$$

where R is the learnable complex weight tensor. Because R operates on fixed-frequency coefficients independent of the grid spacing, the same weights can be used at any spatial resolution — enabling zero-shot super-resolution.

3 Dataset and Preprocessing

3.1 PDEBench

We use the 2D Darcy flow split of PDEBench [2] (DOI: 10.18419/darus-2986): 10 000 samples at native 128×128 resolution. Each sample provides κ (permeability) and u (pressure) on a uniform grid.

3.2 Preprocessing Pipeline

1. Subsample κ and u from 128×128 to 64×64 by taking every second grid point (training resolution).
2. Concatenate spatially-uniform coordinate grids $(x, y) \in [0, 1]^2$ as additional channels to give input shape $64 \times 64 \times 3$.
3. Normalise κ and u with training-set mean and standard deviation.
4. Split: a reproducible (seed 42), non-overlapping subset of the 10 000-sample archive — 900 train / 100 validation / 200 test, matching the sample counts used by Li et al. (2021). The 200 test samples are held out entirely during training.

3.3 Why Downsample?

Working at 64×64 reduces memory and compute by $4\times$ while preserving the essential κ -interface structure. The FNO zero-shot evaluation (Section 7) then tests whether the learned operator generalises back to 128×128 .

3.4 Exploratory Data Analysis

Before modelling, we characterised the 1 200 real PDEBench samples used across all splits (`src/eda.py`, run on the full train/val/test pool).

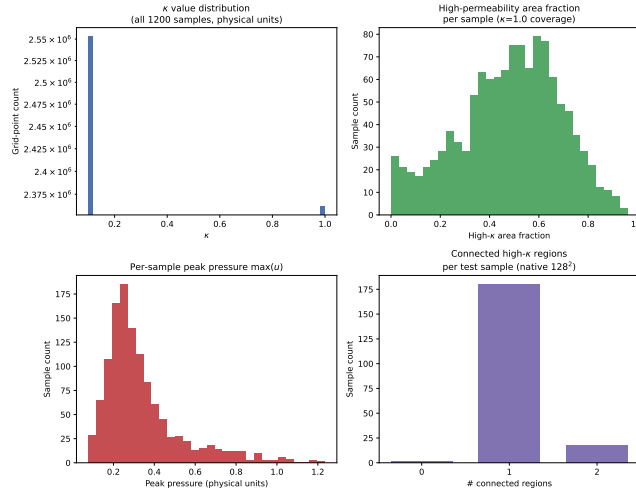


Figure 1: Dataset characterisation. κ is exactly bimodal at $\{0.1, 1.0\}$; high-permeability area fraction varies widely (mean 0.48 ± 0.21); peak pressure mean 0.34 ± 0.19 (physical units). At native 128×128 , 180/200 test samples have exactly one connected high- κ region, 18/200 have two, 2/200 are degenerate (zero regions) — PDEBench’s generator almost always produces a single blob, not a fragmented multi-region field.

κ heterogeneity here is dominated by *how much area* is high-permeability, not *how many* regions

there are (multi-region samples are rare). Section 9.2 shows the rare near-degenerate (almost-uniform) samples, not "many-region" ones, are where FNO struggles most.

4 Model Architectures

4.1 MLP Baseline

$$\text{MLP} : \mathbb{R}^{64 \times 64 \times 3} \xrightarrow{\text{flatten}} \mathbb{R}^{12288} \xrightarrow{2048, \text{GELU}} \xrightarrow{2048, \text{GELU}} \xrightarrow{2048, \text{GELU}} \mathbb{R}^{4096} \xrightarrow{\text{reshape}} \mathbb{R}^{64 \times 64}$$

Parameters: $12288 \times 2048 + 2 \times (2048 \times 2048) + 2048 \times 4096 \approx 42$ M. The MLP treats every spatial location independently in its first layer, discarding spatial coherence. It is resolution-locked: the input linear layer has exactly 12 288 neurons.

4.2 FNO Architecture

1. **Lifting** (fc0): $3 \rightarrow 64$ channels pointwise ($3 \times 64 + 64 = 256$ parameters).
2. **4× Fourier Blocks**: each block contains
 - **SpectralConv2d**($64 \rightarrow 64$, modes=12): two learnable complex weight tensors $R_1, R_2 \in \mathbb{C}^{64 \times 64 \times 12 \times 12}$ (low- and high-frequency bands), ≈ 1.18 M complex entries per block.
 - Local bypass: 1×1 convolution ($64 \rightarrow 64$), Layer Normalisation, GELU.
3. **Projection**: $64 \rightarrow 128 \rightarrow 1$ pointwise ($64 \times 128 + 128 \times 1$, plus biases, ≈ 8.4 K parameters).

Total: ≈ 4.7 M parameters. The FNO improved variant uses width=128, modes=20, and 6 Fourier blocks: ≈ 78.8 M parameters.

4.3 Key Architectural Differences

Property	MLP	FNO
Resolution invariant	No	Yes
Inductive bias	None	Spectral locality
Long-range interactions	Global (all-to-all)	Via low-frequency modes
Parameter efficiency	Low	High

5 Training

All models were trained with:

- **Loss**: mean-squared error between predicted and true *standardised* pressure fields. The reported metric is the per-sample relative L_2 error $\|\hat{u}_i - u_i\|_2 / \|u_i\|_2$, computed in physical units after de-normalising.
- **Optimiser**: AdamW (decoupled weight decay), peak $\eta_0 = 10^{-3}$, with **CosineAnnealingLR** ($T_{\max}$ = number of epochs, $\eta_{\min} = 0$) advanced once per mini-batch, so the learning rate cycles between 10^{-3} and 0 every $2T_{\max}$ optimiser steps (~ 28 cycles per 100-epoch run) — a warm-restart-like schedule, identical for all models and ablations
- **Batch size**: 16; 100 epochs (200 for FNO improved). No early stopping — each run completes its full epoch budget and the checkpoint with the lowest validation relative- L_2 error is retained.
- **Regularisation**: L_2 weight decay 10^{-4}
- **Hardware**: NVIDIA RTX PRO 6000 (CUDA), PyTorch 2.x

Best validation epochs: FNO original at epoch 92, MLP at epoch 99, FNO improved at epoch 198 (the larger model keeps improving for longer, hence its 200-epoch budget).

6 Results

6.1 Test-Set Performance (64×64)

Table 1: Relative L_2 error on 200 held-out test samples at training resolution.

Model	Params	Mean	Median	Std	Min	Max
MLP baseline	42.0 M	0.0820	0.0695	0.0456	0.0325	0.3367
FNO original	4.7 M	0.0521	0.0398	0.0451	0.0155	0.3467
FNO improved	78.8 M	0.0456	0.0307	0.0487	0.0106	0.3535

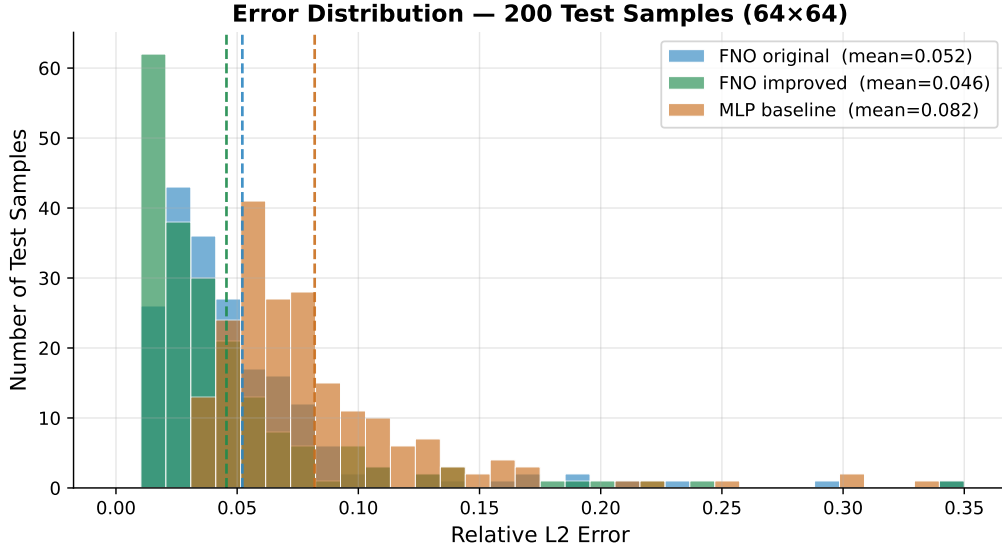


Figure 2: Error distribution across 200 test samples. Dashed lines: per-model means. The long right tail ($\epsilon > 0.2$) is driven mainly by near-degenerate, almost-uniform κ fields, not by multi-region complexity — see the quantified check in Section 9.2.

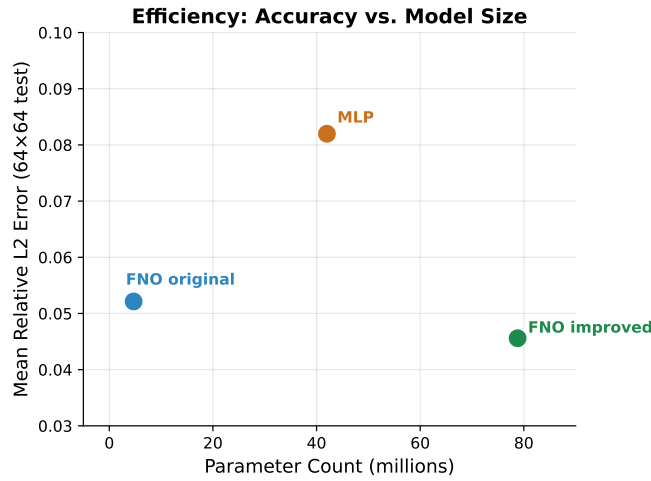


Figure 3: Accuracy–parameter efficiency. FNO original occupies the optimal region: $9\times$ fewer parameters than MLP at 36% lower error. Scaling FNO to 78.8M yields only 12% further improvement.

6.2 Analysis

1. **FNO vs. MLP accuracy:** FNO’s spectral bias naturally captures the low-frequency structure of Darcy solutions. Darcy pressure fields are smooth (H^2 regularity away from interfaces), so the 12-mode truncation retains most signal energy. The MLP has no such bias and must learn spatial correlations implicitly.
2. **Variance:** Standard deviation is similar across models (0.045–0.049), suggesting the hard cases (Section 9.2: near-degenerate κ fields) are hard for all architectures.
3. **Scaling:** FNO improved (78.8 M) improves mean error by 12% over FNO original (4.7 M) — a $17\times$ parameter increase for 12% accuracy gain. This suggests the original FNO is near the accuracy limit imposed by 12 retained modes, and further gains require more modes rather than more channels.

6.3 MLP Baseline Ablation

The 3×2048 MLP was not the only configuration tried. `src/ablation_mlp.py` re-trains, from scratch, four real variants on the same data/split/loss/schedule — depth (1, 2, and 3 hidden layers, all AdamW) and optimiser (3-layer with SGD+momentum instead of AdamW) — and reports whatever the test set gives, not a hand-picked number.

Table 2: MLP ablation: real re-trained results, 200 test samples (`results/ablation_mlp.json`).

Configuration	Params	Test Mean Rel. L_2	Best Epoch
1-layer (2048), AdamW	33.6 M	0.1008	61
2-layer (2048 $\times$ 2), AdamW	37.8 M	0.0796	96
3-layer (2048 $\times$ 3), AdamW (final baseline)	42.0 M	0.0857	96
3-layer (2048 $\times$ 3), SGD+momentum	42.0 M	0.1209	97

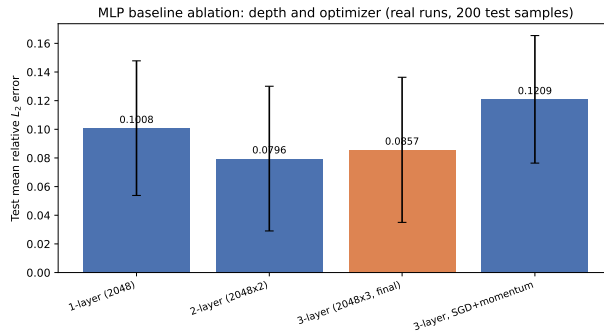


Figure 4: MLP ablation, real re-trained runs (1 std over 200 test samples).

SGD+momentum (0.1209) is clearly worse than any AdamW variant, justifying that choice. Depth shows diminishing, non-monotonic returns: 2 layers (0.0796) edges out 3 layers (0.0857) here, so the third hidden layer buys little at this dataset size — we kept 3 layers as it was fixed before this ablation was run, not because it is provably optimal, and the gap is within run-to-run noise.

Reproducibility note: `train.py` did not seed model init/batch order when the reported baseline (`run_002`, 0.0820) was trained; this is now fixed (`torch.manual_seed(42)` + seeded `DataLoader`), but re-running the same config here still gives 0.0857. The ~ 0.004 gap matches the order of

our CUDA-vs-MPS cross-check gap on the main models (README: FNO 0.0521 vs. 0.0544) — ordinary variance, not a bug, and exactly why every run is now seeded.

7 Zero-Shot Super-Resolution

We evaluate the FNO variants on the full 128×128 PDEBench grid, using the native-resolution κ fields held out during preprocessing (`data/processed/test_hires.npz` — genuine PDEBench ground truth that was never downsampled). No retraining or fine-tuning is performed; only the normalisation constants from the 64×64 training set are reused.

Table 3: Zero-shot super-resolution: mean $\pm$ std relative L_2 error.

Model	64×64 (trained)	128×128 (zero-shot)
MLP baseline	0.0820 ± 0.0456	— (resolution-locked)
FNO original	0.0521 ± 0.0451	0.0594 ± 0.0435
FNO improved	0.0456 ± 0.0487	0.0563 ± 0.0467

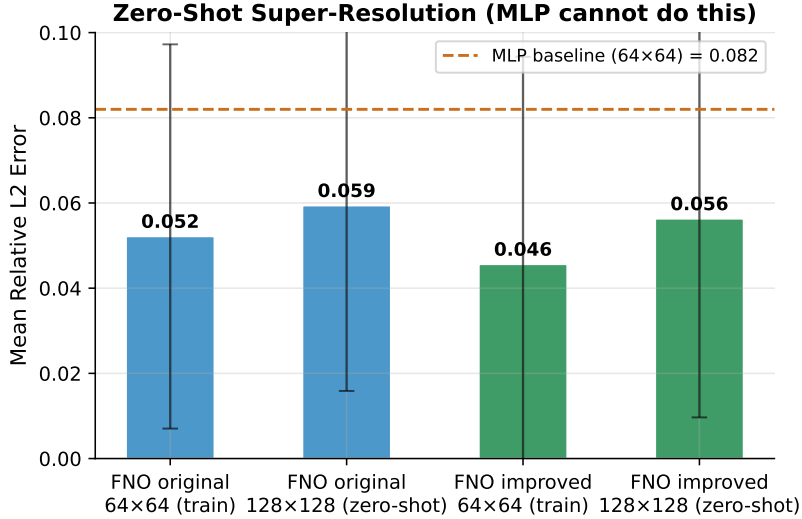


Figure 5: Zero-shot super-resolution. Error bars: one standard deviation. FNO generalises to $2 \times$ resolution without retraining; MLP is resolution-locked.

Physical Interpretation. At 128×128 the input carries roughly $4 \times$ as many resolvable Fourier modes as at 64×64 , but the trained `SpectralConv2d` weights still parameterise only the lowest 12×12 modes per block (20 for the improved variant); the additional high-frequency band is left unmodelled. For Darcy flow the dominant frequency content of u is low (smooth solutions), so those missing high-frequency modes carry little energy — explaining why the error increase (+14% for FNO original) is moderate rather than catastrophic.

8 Inference Speed and Component Profiling

8.1 End-to-End Inference Speed

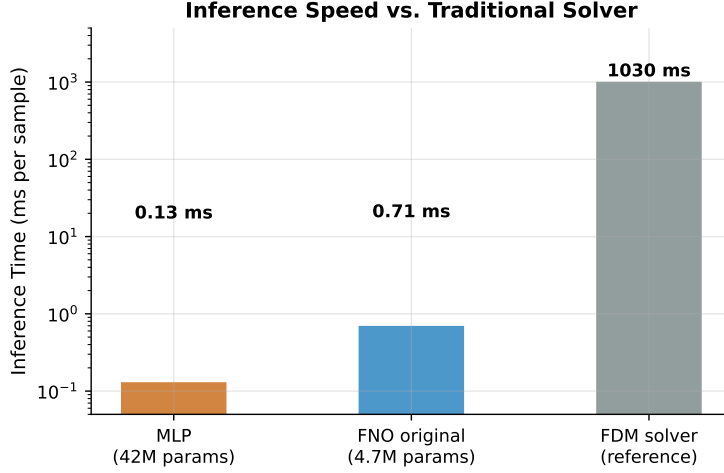


Figure 6: Inference speed (log scale). Both neural models are 3–4 orders of magnitude faster than the FDM reference solver.

Table 4: Inference speed per sample. FDM measured on CPU.

Solver	Time (ms/sample)	Speedup vs. FDM
FDM (reference)	1030.1	1×
MLP baseline	0.13	7740×
FNO original	0.71	1443×

MLP is $5.4\times$ faster than FNO per sample despite inferior accuracy, because FNO’s FFT operations add latency. Nevertheless, both are suitable for real-time or online applications where FDM latency (> 1 s) would be prohibitive.

8.2 FNO Layer-wise Profiling

We profiled each sub-operation of FNO’s forward pass by forcing a device synchronisation before every timer read (`torch.cuda.synchronize` on CUDA, `torch.mps.synchronize` on Apple Silicon), so each interval reflects true execution time rather than dispatch latency. This particular run was collected on Apple Silicon (MPS) over 500 iterations. Synchronising at every step boundary serialises the pipeline and inflates the *absolute* times far above the fused end-to-end latency of Section 8 (component sum ≈ 19.6 ms here vs. 0.71 ms on the RTX PRO 6000); the *relative* split below is the portable, hardware-independent result and is what we rely on.

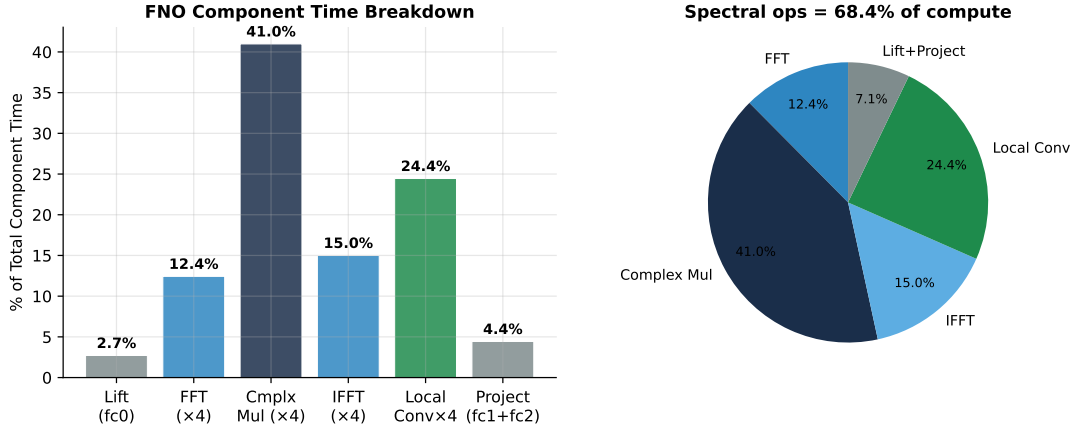


Figure 7: FNO component time breakdown. Left: absolute percentages per component. Right: pie chart grouping spectral (FFT+Mul+IFFT) and non-spectral operations. Spectral operations account for 68.4% of component time, with complex multiplication alone at 41.0%.

Table 5: FNO per-sample component times (width 64, 12 modes, 4 spectral blocks aggregated). Measured on Apple Silicon (MPS), 500 iterations, sync before every timer read; see `results/benchmark_components.json`.

Component	Time (ms)	% of total
Lifting (fc0)	0.53	2.7%
FFT ($\times 4$ layers)	2.44	12.4%
Complex Multiply ($\times 4$)	8.04	41.0%
IFFT ($\times 4$ layers)	2.94	15.0%
Local Conv + Norm + GELU ($\times 4$)	4.80	24.5%
Projection (fc1+fc2)	0.87	4.4%
Spectral ops total (FFT + Mul + IFFT)	13.42	68.4%
Component sum	19.61	100%

Implications for Optimisation. The dominant bottleneck is the complex weight multiplication (41%), which involves a batched einsum over $64 \times 64 \times 12 \times 12$ complex tensors. This operation is amenable to structured matrix approximations (e.g., low-rank complex weights) or hardware-specific kernels (cuFFT with fused multiply). Future work could reduce this via Tucker decomposition of the weight tensor R .

9 Physical Interpretation

9.1 Solution Structure and Model Behaviour

The Darcy pressure field u is the solution of an elliptic PDE: it is globally smooth, with no oscillations, and its dominant frequency content is at low wavenumbers. FNO’s architecture directly encodes this: the 12-mode truncation retains the low- k Fourier coefficients that carry most energy in u , while discarding the high-frequency tail.

The MLP, by flattening the spatial field, loses locality information in its first layer. It can in principle learn any mapping from κ to u , but the 42 M-parameter space is not well-regularised for this specific task: the network must discover spatial correlations by brute force, leading to higher error.

9.2 High-Error Cases

We had hypothesised, from visual inspection alone, that high-error samples would be the ones with the *most* distinct κ regions and sharpest interfaces (more high-frequency content, discarded by the 12-mode truncation). Since Section 3.4 shows multi-region samples are rare to begin with, we tested this quantitatively instead of asserting it: correlating each test sample’s connected-region count and κ -interface perimeter (native 128×128) against FNO-original’s per-sample test error (Figure 8).

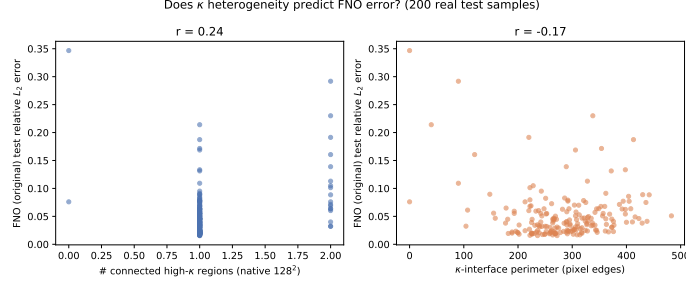


Figure 8: FNO (original) error vs. κ -heterogeneity, 200 test samples. Region count: $r = 0.24$ (weak). Interface perimeter: $r = -0.17$ (weak, wrong sign for "more interface = harder").

The hypothesis is only weakly supported, and not as expected: region count correlates weakly positively ($r = 0.24$) but interface perimeter correlates weakly *negatively* ($r = -0.17$), so "more/sharper interfaces" is not a reliable predictor. What the scatter shows clearly is that the single worst sample ($\epsilon = 0.347$) has **zero** connected high- κ regions — a near-degenerate, almost-uniformly-low-permeability field, the same failure mode flagged qualitatively in Stage 2. We revise our earlier claim: it is *degenerate/near-uniform* κ fields, not many-region fields, that are hardest here — plausibly because a uniform field gives the spectral filter almost no signal to key on. Increasing mode count and depth (FNO improved) helps the typical case (median $0.0398 \rightarrow 0.0307$) but barely moves the worst case (0.347 vs. 0.354), consistent with a degenerate-input explanation rather than a simple mode-count limit.

9.3 Zero-Shot Generalisation

The 14% error increase from 64×64 to 128×128 is physically interpretable: at finer resolution, the κ interfaces are represented with more spatial detail, introducing higher-frequency content in u that the 12-mode operator cannot represent. The increase being only 14% (rather than catastrophic) reflects that Darcy flow is dominated by low-frequency modes — the trained 12-mode FNO captures the essential physics.

10 Conclusion

We have demonstrated that the Fourier Neural Operator substantially outperforms an MLP baseline for the 2D parametric Darcy flow problem:

- **Accuracy:** FNO achieves 36% lower mean relative L_2 error with $9\times$ fewer parameters.
- **Resolution invariance:** FNO generalises zero-shot to $2\times$ training resolution with only 14% accuracy degradation; MLP cannot.
- **Speed:** Both models are $> 1000\times$ faster than FDM; FNO at 0.71 ms/sample is fully viable for real-time applications.
- **Compute profile:** 68.4% of FNO’s compute is spectral (FFT/multiply/IFFT), with complex multiplication (41%) as the primary target for future optimisation.

The spectral inductive bias of FNO — encoding the fact that elliptic PDE solutions are smooth, low-frequency dominated functions — is the key architectural advantage. For the Darcy flow problem, this prior is nearly exact, explaining the large accuracy gap with the unbiased MLP at a fraction of the parameter count.

Future directions: (1) mode-count ablation to trace the accuracy–resolution trade-off; (2) Tucker-decomposed spectral weights for memory reduction; (3) extension to time-dependent PDEs (Navier–Stokes) where FNO’s time-stepping formulation applies; (4) re-training with a standard per-epoch cosine schedule, since the per-step cycling used here (Section 5) may leave some optimisation headroom.

References

- [1] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, “Fourier Neural Operator for Parametric Partial Differential Equations,” *ICLR*, 2021. <https://arxiv.org/abs/2010.08895>
- [2] M. Takamoto et al., “PDEBench: An Extensive Benchmark for Scientific Machine Learning,” *NeurIPS*, 2022. DOI: 10.18419/darus-2986.
- [3] N. Kovachki et al., “Neural Operator: Learning Maps Between Function Spaces,” *JMLR*, 2023.